



Computer Science Department

COMP133 (Spring 2026-1252)

Project Title: Student Record Management System

Phase 2: Structures and Binary File I/O

(Due: Tuesday June 23, 2026 @23:59)

Learning Objectives

- Defining and using **struct** to group related data fields into a single type.
- Replacing parallel arrays with an **array of structs**.
- Binary file I/O for persistent data storage: **loading records at startup and saving on exit**.
- Error handling for file operations (**file not found, read/write failure**).
- Passing structs and struct arrays to functions.

Project Description

In Phase 1 you built the Student Record Management System using parallel arrays. In Phase 2 you will upgrade the system in two key ways:

- (1) Replace the four parallel arrays with a single array of structs, so that each student's data is grouped together in one Student record.
- (2) Use binary file operations. When the program starts it must attempt to load all previously saved students from a binary file called "students.bin". When the user exits, all current students must be written back to the same binary file, making the data persist across sessions.

All other functionality from Phase 1 (menu, add, remove, search, display, validation) must be preserved and adapted to use the struct-based storage.

Requirements

1. Define the Student Structure

Replace the four parallel arrays with the following struct. Every function must work with this struct instead of separate arrays.

```
struct Student {  
    int  studentID;      /* 4-digit unique ID (1000-9999) */  
    char name[50];       /* non-empty, up to 49 characters */  
    int  age;            /* 6 - 50 */  
    double gpa;          /* 0.0 - 4.0 */  
};
```

Global data (in main or as globals shared by all functions):

```
struct Student students[100]; /* array of structs, max 100 */  
int studentCount = 0;         /* number of students stored */
```

2. Binary File: "students.bin"

The file students.bin stores student records in raw binary format. No text formatting or separators are used — the bytes of each struct are written directly.

- File name (hard-coded): students.bin
- Each record is exactly sizeof(struct Student) bytes wide. Transfer of the whole struct array in one or a few calls.

3. Required Functions

All Phase 1 functions must be kept and **updated to use the struct array** and work the same as before. The following file functions will be updated as follows:

1. void initializeStudentRecords () ;

Now performs REAL binary file reading. Called once at program startup.

- Open "students.bin".
- If the file does not exist (fopen returns NULL): print
"No existing records found. Starting with empty system."
- If the file opens successfully: read all records into the students[] array. Update studentCount to the number of records actually read.
- Print "X student record(s) loaded from students.bin." (where X is the number loaded).
- Close the file.

7. void saveStudentRecords () ;

Now performs REAL binary file writing. Called when the user selects Exit.

- Open "students.bin" (creates or overwrites).
- If file open fails: print "Error: Could not save records." and return.
- Write all studentCount records from the students[] array at once.
- Print "X student record(s) saved to students.bin." (where X is studentCount).
- Close the file.

Program Flow (main function outline)

- Call initializeStudentRecords() — opens "students.bin", reads all records into students[].
- Enter a loop:
 - Call displayMainMenu().
 - Read the user's choice (an integer). Validate: if not 1-5, print error and repeat.
 - Choice 1 → addStudent()
 - Choice 2 → removeStudent()
 - Choice 3 → searchStudent()
 - Choice 4 → displayAllStudents()
 - Choice 5 → call saveStudentRecords(), then break out of the loop.
- Return 0.

Very Important Notes

- **Real binary file operations are required in this phase.** Using printf messages only (as in Phase 1) will receive a grade of 0 for the file section.
- **All Phase 1 functionality must still work.** The only structural change is using a struct array and real file I/O.
- **Submit** both your main.c file and a sample students.bin binary file containing at least 5 pre-loaded students. Turn in under COMPUTER AND PROGRAMMING-Lecture-1252-COMP133-Meta on ITC.
- **You must include your full name, student ID number, and lecture + lab section numbers in a comment at the beginning of your main.c file.**
- **Late submissions or those not turned in on ITC will not be accepted for any reason.**
- **Code similarity and AI will be checked.** Any proved similarity or AI-generated code will be treated as cheating with a grade of 0, and academic honesty bylaws will be applied.
- **You are not allowed to use any library function or feature not covered in class.**
- **Make sure your program works very similar to the Sample Run below.** In particular, the persistence test (run the program twice — second run must show the data saved in the first run) must work.

Sample Run

The sample run below shows two consecutive executions to demonstrate data persistence.

— First execution (no previous data file) —

```
No existing records found. Starting with empty system.
=== STUDENT RECORD MANAGEMENT SYSTEM ===
1- Add New Student
2- Remove Student
3- Search for Student
4- Display All Students
5- Exit System
Enter your choice (1-5): 1

=== ADD NEW STUDENT ===
Enter 4-digit Student ID (1000-9999): 1001
Enter Student name: Muna Samara
Enter Student Age: 19
Enter GPA: 3.5
Student added successfully!

=== STUDENT RECORD MANAGEMENT SYSTEM ===
1- Add New Student
2- Remove Student
3- Search for Student
4- Display All Students
5- Exit System
Enter your choice (1-5): 1

=== ADD NEW STUDENT ===
Enter 4-digit Student ID (1000-9999): 1002
```

Enter Student name: Basem Alawi
Enter Student Age: 21
Enter GPA: 2.8
Student added successfully!

=== STUDENT RECORD MANAGEMENT SYSTEM ===

- 1- Add New Student
- 2- Remove Student
- 3- Search for Student
- 4- Display All Students
- 5- Exit System

Enter your choice (1-5): 4

=== ALL STUDENTS ===

ID	Name	Age	GPA
1001	Muna Samara	19	3.50
1002	Basem Alawi	21	2.80

Total Students: 2

Average GPA: 3.15

=== STUDENT RECORD MANAGEMENT SYSTEM ===

- 1- Add New Student
- 2- Remove Student
- 3- Search for Student
- 4- Display All Students
- 5- Exit System

Enter your choice (1-5): 3

=== SEARCH STUDENT ===

Enter Student ID to search: 1003

Student not found.

=== STUDENT RECORD MANAGEMENT SYSTEM ===

- 1- Add New Student
- 2- Remove Student
- 3- Search for Student
- 4- Display All Students
- 5- Exit System

Enter your choice (1-5): 2

=== REMOVE STUDENT ===

Enter Student ID to remove: 1002

Removing Student: Basem Alawi (ID: 1002)

Student removed successfully!

=== STUDENT RECORD MANAGEMENT SYSTEM ===

- 1- Add New Student
- 2- Remove Student
- 3- Search for Student
- 4- Display All Students
- 5- Exit System

Enter your choice (1-5): 5

1 student record(s) saved to students.bin.

Thank you for using Student Record Management System!

— Second execution (loads data saved in first run) —

1 student record(s) loaded from students.bin.

=== STUDENT RECORD MANAGEMENT SYSTEM ===

1- Add New Student

2- Remove Student

3- Search for Student

4- Display All Students

5- Exit System

Enter your choice (1-5): 4

=== ALL STUDENTS ===

ID	Name	Age	GPA
1001	Muna Samara	19	3.50

Total Students: 1

Average GPA: 3.50

=== STUDENT RECORD MANAGEMENT SYSTEM ===

1- Add New Student

2- Remove Student

3- Search for Student

4- Display All Students

5- Exit System

Enter your choice (1-5): 3

=== SEARCH STUDENT ===

Enter Student ID to search: 1001

Student Found:

Student ID: 1001

Student Name: Muna Samara

Student Age: 19

GPA: 3.50

=== STUDENT RECORD MANAGEMENT SYSTEM ===

1- Add New Student

2- Remove Student

3- Search for Student

4- Display All Students

5- Exit System

Enter your choice (1-5): 5

1 student record(s) saved to students.bin.

Thank you for using Student Record Management System!